

智能移动开发12周

Add repository and Manual DI

English

1 Before you begin

2 **Separate the UI layer and Data layer**

3 Create Data layer

4 Dependency injection

5 Attach application container to the app

6 Add repository to ViewModel

7 Get setup for local tests

8 Create fake data and dependencies for tests

[Report a mistake](#)

The UI layer's concern is displaying the data it is provided. The UI no longer retrieves the data as that is the data layer's concern.

The data layer is made up of one or more repositories. Repositories themselves contain zero or more data sources.

```
graph TD;
    UI[UI Layer] --> Domain[Domain Layer (optional)];
    Domain --> Data[Data Layer];
    subgraph DataLayer [Data Layer]
        Repositories[Repositories] --> DataSources[Data Sources];
    end
```

Best practices require the app to have a repository for each type of data source your app uses.

In this codelab, the app has one data source, so it has one repository after you refactor the code. For this app, the repository that retrieves data from the internet completes the data source's responsibilities. It does so by making a network request to an API. If the data source coding is more complex or additional data sources are added, the data source responsibilities are encapsulated in separate data source classes, and the repository is responsible for

Back

Next

Add repository and Manual DI

English

1 Before you begin

2 **Separate the UI layer and Data layer**

3 Create Data layer

4 Dependency injection

5 Attach application container to the app

6 Add repository to ViewModel

7 Get setup for local tests

8 Create fake data and dependencies for tests

[Report a mistake](#)

- Resolves conflicts between multiple data sources.
- Abstracts sources of data from the rest of the app.
- Contains business logic.

The **Mars Photos** app has a single data source, which is the network API call. It does not have any business logic, as it is just retrieving data. The data is exposed to the app through the repository class, which abstracts away the source of the data.

```
graph TD;
    UI[UI Layer] --> Domain[Domain Layer (optional)];
    Domain --> Data[Data Layer];
    subgraph DataLayer [Data Layer]
        Repositories[Repositories] --> DataSources[Data Sources];
    end
```

Back

Next

1 Before you begin

2 Separate the UI layer and Data layer

3 Create Data layer

4 Dependency injection

5 Attach application container to the app

6 Add repository to ViewModel

7 Get setup for local tests

8 Create fake data and dependencies for tests

 [Report a mistake](#)

Back

Next, you need to update the `ViewModel` code to use the repository to get the data as Android best practices suggest.

9. Open the `ui/screens/MarsViewModel.kt` file.

10. Scroll down to the `getMarsPhotos()` method.

11. Replace the line "`val listResult = MarsApi.retrofitService.getPhotos()`" with the following code:

```
import com.example.marsphotos.data.NetworkMarsPhotosRepository

val marsPhotosRepository = NetworkMarsPhotosRepository()
val listResult = marsPhotosRepository.getMarsPhotos()
```

```
private fun getMarsPhotos() {
    viewModelScope.launch { this: CoroutineScope
        marsUiState = try {
            val marsPhotosRepository = NetworkMarsPhotosRepository()
            val listResult = marsPhotosRepository.getMarsPhotos()
            MarsUiState.Success(photos = "${listResult.size} Mars photos retrieved")
        } catch (e: IOException) {
            MarsUiState.Error
        } catch (e: HttpException) {
            MarsUiState.Error
        }
    }
}
```

12. Run the app. Notice that the results displayed are the same as the previous results.

Next

1 Before you begin

2 Separate the UI layer and Data layer

3 Create Data layer

4 Dependency injection

5 Attach application container to the app

6 Add repository to ViewModel

7 Get setup for local tests

8 Create fake data and dependencies for tests

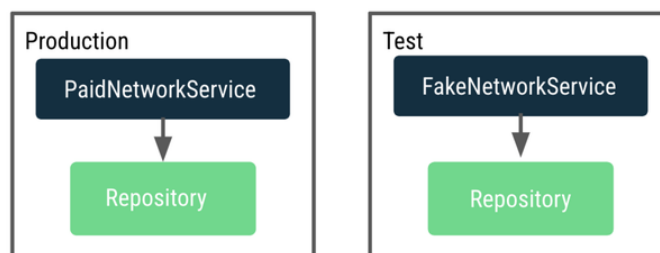
 [Report a mistake](#)

Back

Implementing dependency injection:

- **Helps with the reusability of code.** Code is not dependent on a specific object, which allows for greater flexibility.
- **Makes refactoring easier.** Code is loosely coupled, so refactoring one section of code does not impact another section of code.
- **Helps with testing.** Test objects can be passed in during testing.

One example of how DI can help with testing is when testing the network calling code. For this test, you are really trying to test that the network call is made and that data is returned. If you had to pay each time you made a network request during a test, you might decide to skip testing this code, as it can get expensive. Now, imagine if we can fake the network request for testing. How much happier (and wealthier) does that make you? For testing, you can pass a test object to the repository that returns fake data when called without actually performing a real network call.



Next

1 Before you begin

2 Separate the UI layer and Data layer

3 Create Data layer

4 **Dependency injection**

5 Attach application container to the app

6 Add repository to ViewModel

7 Get setup for local tests

8 Create fake data and dependencies for tests

 [Report a mistake](#)

Back

Next

```
override suspend fun getPhotos(): List<Photo> {  
    return repository.getPhotos()  
}
```

12. Remove the following import from the `MarsPhotosRepository.kt` file.

```
// Remove  
import com.example.marsphotos.network.MarsApi
```

From the `network/MarsApiService.kt` file, we moved all the code out of the object. We can now delete the remaining object declaration as it is no longer needed.

13. Delete the following code:

```
object MarsApi {  
  
}
```

1 Before you begin

2 Separate the UI layer and Data layer

3 Create Data layer

4 Dependency injection

5 **Attach application container to the app**

6 Add repository to ViewModel

7 Get setup for local tests

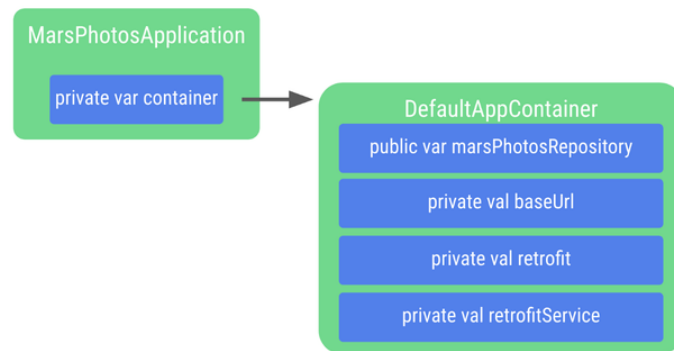
8 Create fake data and dependencies for tests

 [Report a mistake](#)

Back

Next

The steps in this section connect the application object to the application container as shown in the following figure.

1. Right-click on `com.example.marsphotos` and select **New > Kotlin Class/File**.2. In the dialog, enter `MarsPhotosApplication`. This class inherits from the application object, so you need to add it to the class declaration.

```
import android.app.Application
```

1 Before you begin

2 Separate the UI layer and Data layer

3 Create Data layer

4 Dependency injection

5 **Attach application container to the app**

6 Add repository to ViewModel

7 Get setup for local tests

8 Create fake data and dependencies for tests

 [Report a mistake](#)

Back

```
        container = DefaultAppContainer()
    }
```

4. The complete `MarsPhotosApplication.kt` file should look like the following code:

```
package com.example.marsphotos

import android.app.Application
import com.example.marsphotos.data.AppContainer
import com.example.marsphotos.data.DefaultAppContainer

class MarsPhotosApplication : Application() {
    lateinit var container: AppContainer
    override fun onCreate() {
        super.onCreate()
        container = DefaultAppContainer()
    }
}
```

5. You need to update the Android manifest so the app uses the application class you just defined. Open the `manifests/AndroidManifest.xml` file.

Android

Next

1 Before you begin

2 Separate the UI layer and Data layer

3 Create Data layer

4 Dependency injection

5 Attach application container to the app

6 Add repository to ViewModel

7 Get setup for local tests

8 **Create fake data and dependencies for tests** [Report a mistake](#)

Back

On this object, create a property set to a list of `MarsPhoto` objects. The list does not have to be long, but it should contain at least two objects.

Note: For the purposes of the tests you will write in this codelab, the data stored in the objects does not necessarily need to be representative of the data that the API returns. In other words, you don't need to include valid IDs or URLs.

```
object FakeDataSource {

    const val idOne = "img1"
    const val idTwo = "img2"
    const val imgOne = "url.1"
    const val imgTwo = "url.2"
    val photosList = listOf(
        MarsPhoto(
            id = idOne,
            imgSrc = imgOne
        ),
        MarsPhoto(
            id = idTwo,
            imgSrc = imgTwo
        )
    )
}
```

Next

[1 Before you begin](#)[2 Separate the UI layer and Data layer](#)[3 Create Data layer](#)[4 Dependency injection](#)[5 Attach application container to the app](#)[6 Add repository to ViewModel](#)[7 Get setup for local tests](#)[8 Create fake data and dependencies for tests](#)[Report a mistake](#)[Back](#)

4. Create a `TestDispatcher` constructor parameter for the `TestDispatcherRule`.

This parameter enables the use of different dispatchers, such as `StandardTestDispatcher`. This constructor parameter needs to have a default value set to an instance of the `UnconfinedTestDispatcher` object. The `UnconfinedTestDispatcher` class inherits from the `TestDispatcher` class and it specifies that tasks must not be executed in any particular order. This pattern of execution is good for simple tests as coroutines are handled automatically. Unlike `UnconfinedTestDispatcher`, the `StandardTestDispatcher` class enables full control over coroutine execution. This way is preferable for complicated tests that require a manual approach, but it is not necessary for the tests in this code lab.

```
class TestDispatcherRule(  
    val testDispatcher: TestDispatcher = UnconfinedTestDispatcher(),  
) : TestWatcher() {  
  
}
```

5. The primary goal of this test rule is to replace the `Main` dispatcher with a test dispatcher before a test begins to execute. The `starting()` function of the `TestWatcher` class executes before a given test executes. Override the `starting()` function.

```
class TestDispatcherRule(  
    val testDispatcher: TestDispatcher = UnconfinedTestDispatcher(),  
)
```

[Next](#)